UNIVERSIDAD DEL VALLE DE GUATEMALA

Computación paralela y distribuida - 40

Marlon Fuentes



Proyecto 2

Búsqueda distribuida de claves DES con OpenMPI

Grupo 6

Ihan Marroquin - 23108

Luis Gonzalez - 23353

Diego Patzan - 23525

GUATEMALA, 14 DE OCTUBRE DEL 2026

Resumen

El proyecto implementa en C11 el cifrado y descifrado DES de archivos, una búsqueda secuencial y una búsqueda paralela con OpenMPI. El programa acepta un archivo cifrado, una palabra conocida, un intervalo de índices y una política de distribución. El proceso raíz valida la entrada, distribuye el ciphertext y coordina reportes en tandas; todos los procesos prueban candidatos y terminan mediante una decisión colectiva. La solución recuperó exactamente el caso del equipo y superó pruebas con vectores conocidos, archivos binarios, rangos límite, entradas inválidas y búsquedas sin coincidencia.

La evaluación incluye 30 mediciones del caso del equipo y 210 mediciones controladas para comparar reparto contiguo y cíclico con 2, 4 y 6 procesos. El reparto cíclico produjo resultados más consistentes entre posiciones de clave. El reparto por bloques fue muy rápido cuando la clave coincidió con el inicio del bloque de un proceso, pero también fue más lento que la versión secuencial en otros casos. Esto confirma que la ubicación de la llave puede inflar o degradar el speedup y que una sola medición no basta para evaluar el algoritmo.

Índice

Resumen....2

Índice....2

1 Introducción....3

2 Antecedentes de DES y fuerza bruta....3

3 Diseño e implementación....4

3 1 Formato del archivo y caso del equipo....4

3 2 Cifrado y descifrado....4

3 3 memcpy y búsqueda de substring....4

3 4 Comunicación MPI....4

3 5 Reparto por bloques y reparto cíclico....5

3 6 Programación defensiva....5

4 Flujo del programa....5

5 Metodología experimental....6

6 Resultados....7

6 1 Caso del equipo....7

6 2 Posición de clave y distribución....7

6 3 Pruebas de otros grupos....8

7 Retos encontrados....8

8 Conclusiones y recomendaciones....8

9 Referencias....9

10 Anexo 1 Catálogo de funciones 10
11 Anexo 2 Bitácora de pruebas 11
12 Anexo 3 Coevaluación 12

1 Introducción

DES es un caso didáctico útil para estudiar fuerza bruta porque su espacio efectivo contiene 2^56 claves y cada candidato puede comprobarse de forma independiente. Esa independencia permite distribuir el intervalo entre procesos, aunque la coordinación, el tamaño de tanda y la posición de la clave condicionan el rendimiento observado. El objetivo fue construir una solución reproducible que cifrara un texto, recuperara su clave mediante búsqueda secuencial y paralela, validara los datos de entrada y registrara tiempos comparables.

El repositorio contiene el código propio del equipo, un formato binario documentado, pruebas automatizadas, datos de medición y herramientas de empaquetado. OpenSSL provee la primitiva DES-ECB; el equipo implementó la representación de índices de 56 bits, el manejo de archivos, la validación PKCS#7, la búsqueda de la palabra y el protocolo MPI.

2 Antecedentes de DES y fuerza bruta

DES es un cifrador simétrico por bloques. La misma clave se usa para cifrar y descifrar. Opera sobre bloques de 64 bits y representa la clave en 64 bits, aunque ocho se reservan para paridad y solo 56 intervienen en la transformación. El estándar FIPS 46-3 especifica DES y TDEA. NIST retiró DES en 2005 porque dejó de ofrecer la seguridad necesaria; en este proyecto se usa exclusivamente como ejercicio académico [1][2].

La transformación aplica una permutación inicial, divide el bloque en dos mitades de 32 bits y ejecuta 16 rondas Feistel. En cada ronda, la mitad derecha alimenta una función que expande 32 bits a 48, combina una subclave mediante XOR, reduce el resultado con ocho cajas S y aplica una permutación. El calendario de clave genera 16 subclaves de 48 bits. El descifrado usa la misma estructura en el orden inverso de subclaves [1].

El modo ECB cifra cada bloque independientemente. En el formato del proyecto, PKCS#7 completa el último bloque: si faltan k bytes se agregan k copias del valor k; si la entrada ya está alineada se agrega un bloque completo. El archivo resultante no contiene cabecera, sal ni vector de inicialización. ECB no debe usarse para proteger información real porque conserva patrones entre bloques iguales; aquí facilita una prueba controlada e interoperable con la herramienta incluida.

La fuerza bruta enumera índices en [0, 2^56). Cada índice se convierte en ocho grupos de siete bits; cada grupo ocupa los bits altos de un byte y el bit bajo se ajusta para paridad impar. Para comprobar un candidato, el programa descifra, valida el relleno y busca una palabra conocida dentro de una longitud explícita. La coincidencia es una clave posible; en los casos de prueba se verifica además que el archivo recuperado sea idéntico al original.

3 Diseño e implementación

3 1 Formato del archivo y caso del equipo

data/texto.bin contiene 48 bytes DES-ECB con PKCS#7. El texto original contiene 40 bytes y la palabra literal de reconocimiento es Estudiante. La pista humana publicada es “Le tiene miedo a las mujeres”. El repositorio no publica la clave ni el texto original; el docente puede recibirlos por separado para verificar la recuperación.

3 2 Cifrado y descifrado

des_encrypt_buffer convierte el índice en una clave DES con paridad, inicializa un contexto EVP y cifra la entrada con relleno. des_decrypt_buffer rechaza longitudes que no sean múltiplos de ocho, descifra y valida PKCS#7. La API EVP permite reutilizar contextos y exige comprobar el valor de retorno de cada inicialización, actualización y finalización [3]. DES se obtiene del proveedor legacy de OpenSSL 3 [4].

La rutina des_candidate_try desempeña la función de tryKey: recibe el índice, el ciphertext, la palabra y sus longitudes; descifra en un buffer reutilizable; valida el relleno; y llama a des_contains_bytes. Devuelve 1 para coincidencia, 0 para candidato rechazado y -1 para error operativo.

3 3 memcpy y búsqueda de substring

memcpy copia un número explícito de bytes entre regiones que no se solapan. El programa la usa para copiar la palabra con su terminador NUL y para colocar el reporte local del proceso raíz en la matriz de reportes. Los tamaños provienen de buffers ya reservados.

El programa base proponía strstr, que trabaja con cadenas terminadas en NUL. Un plaintext puede contener bytes cero y no garantiza un terminador dentro del buffer; por eso la implementación usa des_contains_bytes, que recorre solo posiciones válidas y compara con memcmp. Conserva el propósito de buscar un substring, sin leer fuera de límites ni truncar datos binarios.

3 4 Comunicación MPI

El rank 0 valida argumentos y lee el archivo. MPI_Bcast distribuye el estado, los límites, la política, la palabra y el ciphertext. MPI_Allreduce impide que la búsqueda comience si algún proceso no pudo reservar o inicializar sus recursos.

Antes de cada tanda, rank 0 publica un MPI_Irecv por cada trabajador. Esta operación inicia una recepción no bloqueante y devuelve una solicitud [5]. Los trabajadores prueban hasta batch candidatos y envían un reporte de cuatro enteros con MPI_Send, una operación bloqueante de modo estándar [6]. Rank 0 completa cada solicitud con MPI_Wait antes de leer el buffer [7]. Después elige la menor clave reportada y difunde una de cuatro decisiones: continuar, encontrada, agotada o error.

MPI_Reduce suma los intentos y obtiene el máximo de los tiempos locales. Rank 0 verifica nuevamente la clave, escribe el archivo recuperado si se solicitó y difunde el código de salida. Todos liberan los contextos y terminan con MPI_Finalize.

3 5 Reparto por bloques y reparto cíclico

El modo block divide [inicio, fin) en segmentos contiguos, distribuye el residuo entre los primeros ranks y hace que cada proceso avance de uno en uno dentro de su segmento. El modo cyclic asigna al rank r los candidatos inicio + r + j×p. Ambos cubren el intervalo una sola vez. El modo se selecciona con --distribution block|cyclic; el valor predeterminado es cyclic.

El reparto por bloques puede resultar excepcional si la clave está cerca del inicio de un segmento, pero puede obligar a un proceso a recorrer gran parte de su bloque mientras otros quedan ociosos. El reparto cíclico entrelaza posiciones y reduce la dependencia entre una región numérica y un rank específico.

3 6 Programación defensiva

La CLI rechaza opciones desconocidas, duplicadas, vacías o fuera de [0, 2^56]. También valida que el ciphertext no esté vacío, que contenga bloques completos, que las longitudes quepan en los contadores MPI y que la salida no sea el mismo archivo de entrada. Las escrituras se hacen mediante archivo temporal y reemplazo. Los errores MPI se convierten en mensajes con el rank y la operación. Ningún proceso entra en una colectiva que dependa de recursos si otro proceso falló al prepararlos.

4 Flujo del programa

1. Inicializar MPI y obtener rank y cantidad de procesos.
2. En rank 0, capturar argumentos, validar rutas, palabra, límites, tanda y distribución.
3. Leer texto.bin y comprobar su tamaño.
4. Difundir estado de preparación; terminar de forma uniforme ante ayuda o error.
5. Difundir parámetros, palabra y ciphertext.
6. Inicializar OpenSSL y buffers en cada rank; comprobar disponibilidad global.
7. Calcular el primer candidato, el límite local y el salto según block o cyclic.
8. Publicar recepciones no bloqueantes en rank 0.
9. Probar una tanda de candidatos en cada proceso.
10. Enviar reportes; esperar las recepciones; resolver encontrada, agotada o error.
11. Difundir la decisión. Repetir si corresponde.
12. Reducir intentos y tiempo máximo.
13. Verificar la clave en rank 0, imprimir archivo, palabra, clave posible y métricas.
14. Liberar recursos y finalizar MPI.

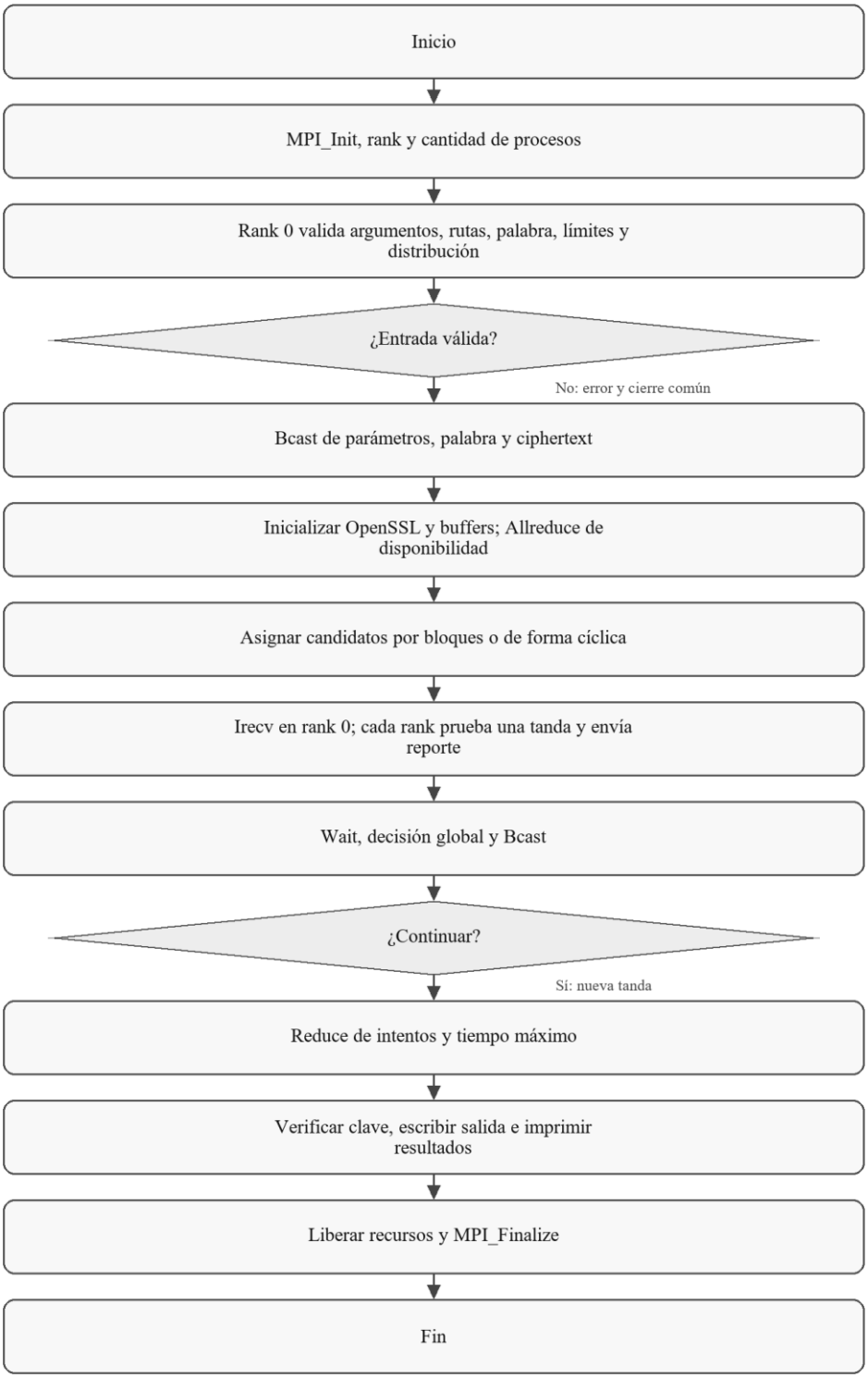


Figura 1 Flujo completo de la búsqueda MPI

5 Metodología experimental

Las mediciones se ejecutaron dentro de Ubuntu 24.04 en Docker Desktop sobre Windows. El contenedor utilizó GCC 13.3, OpenMPI 4.1.6 y OpenSSL 3.0.13. El código se compiló con -

O3 -std=c11 -Wall -Wextra -Wpedantic sin advertencias. Cada configuración tuvo un calentamiento excluido y diez repeticiones medidas.

El tiempo secuencial usa reloj monotónico. En MPI, cada rank usa MPI_Wtime y el resultado es el máximo de los tiempos locales. La lectura y la preparación criptográfica se excluyen; las tandas y su comunicación se incluyen. Cada corrida verifica la recuperación completa.

Para el caso del equipo se usó [0, 2^20) y la misma clave en todas las ejecuciones. Para estudiar la dependencia posicional se generaron tres casos controlados en el mismo espacio escalado: fácil en 1/2 + 1, media en 1/2 + 1/8 y difícil en ceil(1/7 + 1/13) de 2^20. Estas posiciones reproducen las proporciones del enunciado sin afirmar que se recorrió 2^56.

6 Resultados

6 1 Caso del equipo

Configuración	Procesos	Repeticiones	Promedio s	Speedup	Eficiencia
Secuencial	1	10	0.507332	1.000	100.0%
MPI cíclico	2	10	0.296564	1.711	85.5%
MPI cíclico	4	10	0.180405	2.812	70.3%

Las 30 ejecuciones recuperaron los 40 bytes originales. La mejora aumentó con cuatro procesos, pero la eficiencia disminuyó por comunicación, sincronización y recursos compartidos.

6 2 Posición de clave y distribución

Caso	Posición normalizada	Política	p	Promedio s	Speedup	Eficiencia
Fácil	0.500001	Block	4	0.010907	93.138	2328.5%
Fácil	0.500001	Cyclic	4	0.348808	2.912	72.8%
Media	0.625000	Block	4	0.339026	3.621	90.5%
Media	0.625000	Cyclic	4	0.419573	2.926	73.2%
Difícil	0.219781	Block	4	0.584252	0.732	18.3%
Difícil	0.219781	Cyclic	4	0.154094	2.777	69.4%

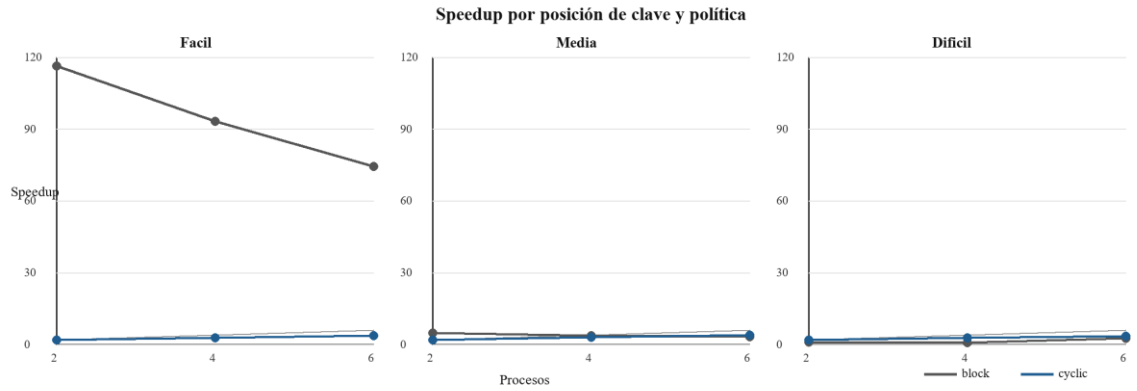


Figura 2 Speedup según posición y política de distribución

La eficiencia aparente superior a 100% en el caso fácil no representa una máquina que haga más trabajo que sus recursos. La clave cayó casi al inicio del bloque de un trabajador y la búsqueda paralela evitó la mayoría de candidatos que recorrió la secuencial. En el caso difícil, block con cuatro procesos fue más lento que la secuencial; cyclic mantuvo un speedup de 2.777. La bitácora completa contiene 210 filas y el resumen 21 configuraciones.

6 3 Pruebas de otros grupos

El programa acepta cualquier ciphertext que utilice el formato documentado y permite registrar nuevos casos con el mismo procedimiento. En los insumos recibidos solo estaba el ciphertext del Grupo 6, idéntico al archivo ya versionado. No se presentan dos casos externos ni se atribuyen resultados a otros equipos porque esos archivos y sus palabras literales no fueron proporcionados. Esta limitación debe resolverse antes de entregar si el docente exige evidencia de dos publicaciones ajenas.

7 Retos encontrados

El primer reto fue representar 2^56 candidatos sin repetir variantes que solo difieren en paridad. Se resolvió separando el índice en ocho grupos de siete bits y calculando paridad impar. El segundo fue distinguir una clave posible sin tratar bytes arbitrarios como una cadena C; se resolvió con longitud explícita, validación PKCS#7 y búsqueda binaria acotada.

En MPI, el riesgo principal era un bloqueo cuando algunos procesos agotaran su intervalo antes que otros. La solución obliga a cada rank agotado a seguir reportando su estado y participando en colectivas hasta la decisión global. Las recepciones se publican antes de que rank 0 busque su tanda y todos los buffers viven hasta completar MPI_Wait.

Finalmente, los speedups dependían de la posición. Implementar dos políticas y tres posiciones controladas permitió separar una mejora reproducible de una coincidencia favorable. Los resultados mostraron que reportar solo el mejor caso conduciría a una conclusión incorrecta.

8 Conclusiones y recomendaciones

La solución cumple el recorrido funcional de cifrado, descifrado y búsqueda DES en versiones secuencial y MPI. Las pruebas automatizadas cubren el vector DES conocido, entradas vacías y binarias, límites del dominio, ausencia de coincidencia, validación de argumentos y

terminación sin bloqueo. Todas las mediciones reportadas corresponden a ejecuciones verificadas.

El reparto cíclico es la política recomendada como valor predeterminado porque sostuvo mejoras entre 2 y 6 procesos en las tres posiciones. El reparto por bloques debe conservarse como comparación: puede ganar por mucho o perder frente a la versión secuencial según dónde caiga la clave.

Se recomienda presentar promedios, speedup, eficiencia, cantidad de procesos, intervalo, distribución y posición normalizada. También se recomienda mantener DES solo con fines académicos, usar algoritmos actuales para protección real y obtener dos desafíos externos compatibles para cerrar el requisito comparativo sin fabricar evidencia.

9 Referencias

[1] National Institute of Standards and Technology. *FIPS 46-3 Data Encryption Standard*. 1999. https://csrc.nist.gov/pubs/fips/46-3/final

[2] National Institute of Standards and Technology. *Withdrawal of FIPS 46-3, FIPS 74 and FIPS 81*. 2005. https://csrc.nist.gov/news/2005/withdrawal-of-fips-46-3-fips-74-and-fips-81

[3] OpenSSL Project. *EVP cipher routines*. OpenSSL 3.0 documentation. https://docs.openssl.org/3.0/man3/EVP_EncryptInit/

[4] OpenSSL Project. *OpenSSL legacy provider*. OpenSSL 3.0 documentation. https://docs.openssl.org/3.0/man7/OSSL_PROVIDER-legacy

[5] Open MPI Project. *MPI_Irecv*. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Irecv.3.html

[6] Open MPI Project. *MPI_Send*. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Send.3.html

[7] Open MPI Project. *MPI_Wait*. https://docs.open-mpi.org/en/main/man-openmpi/man3/MPI_Wait.3.html

10 Anexo 1 Catálogo de funciones

Función	Entradas	Salidas	Descripción
des_crypto_init	estructura, buffer de error	1 o 0	Carga proveedores y obtiene DES-ECB.
des_crypto_cleanup	estructura criptográfica	ninguna	Libera cipher y proveedores.
des_parse_index	texto, máximo	índice	Convierte decimal con validación estricta.
des_read_key_file	ruta	índice de clave	Lee una clave decimal privada.
des_read_file	ruta	buffer y longitud	Lee bytes con límite y errores.
des_write_file	ruta, bytes, longitud	1 o 0	Escribe mediante temporal y reemplazo.
des_index_to_key	índice uint64	clave de 8 bytes	Mapea 56 bits y calcula paridad impar.
des_encrypt_buffer	contexto, índice, entrada	ciphertext y longitud	Cifra DES-ECB con PKCS#7.
des_decrypt_buffer	contexto, índice, ciphertext	plaintext y longitud	Descifra y valida relleno.
des_candidate_init	crypto, longitud	candidato	Reserva contexto y buffer reutilizable.
des_candidate_try	candidato, índice, datos, keyword	1, 0 o -1	Prueba una clave sin modificar el ciphertext.
des_contains_bytes	datos y keyword con longitudes	booleano	Busca substring binario dentro de límites.
des_monotonic_seconds	ninguna	double	Obtiene tiempo monotónico secuencial.
parse_options	argc, argv	opciones o error	Valida argumentos MPI y distribución.
same_file	dos rutas	booleano	Detecta si entrada y salida son el mismo archivo.
mpi_check	código MPI, operación, rank	ninguna	Informa errores MPI y aborta de forma controlada.
main en bruteforce.c	argumentos CLI	código de salida	Coordina lectura, difusión, búsqueda, reportes y cierre.
run_cipher	operación y opciones	código de salida	Ejecuta cifrado o descifrado secuencial.
run_search	opciones	código de salida	Ejecuta fuerza bruta secuencial y reporta métricas.

11 Anexo 2 Bitácora de pruebas

La bitácora evidence/distribution_benchmark.csv contiene caso, posición normalizada, algoritmo, distribución, procesos, repetición, segundos, intentos, corrección y commit. evidence/distribution_summary.csv contiene los 21 promedios con speedup y eficiencia. scripts/validate_benchmark.py recalcula los valores y exige 210 recuperaciones correctas.

evidence/verificacion.log registra el vector público DES, roundtrips de archivo vacío, longitud irregular y binario con NUL; búsquedas encontrada y agotada; intervalos vacíos; límite 2^56 - 1; argumentos inválidos; ambos repartos; y un caso de 40001 candidatos que termina sin bloqueo.

12 Anexo 3 Coevaluación

Integrante	Aporte principal	Evaluación
Diego Patzan	Distribución MPI por bloques y cíclica, automatización y validación de pruebas	Cumplió con el trabajo asignado y verificó sus cambios.
Ihan Marroquin	Diseño de mediciones, análisis de resultados y documentación técnica	Cumplió con el trabajo asignado y verificó sus cambios.
Luis Gonzalez	Integración del caso del equipo, revisión final y preparación de entrega	Cumplió con el trabajo asignado y verificó sus cambios.

Los tres integrantes revisaron el flujo completo y autorizaron el uso de sus identidades Git en la computadora compartida.